

inframon

교량 InSAR → PINN 가상센싱 → 위험도 → IFC 디지털 트윈

사용자 워크플로우 안내서

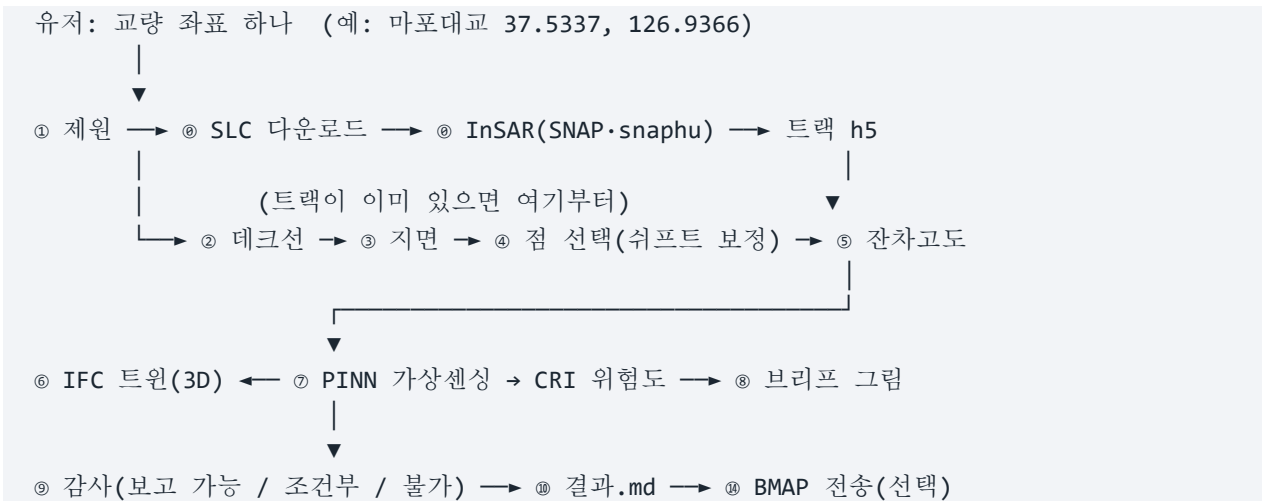
2026-09-09 · <https://github.com/tjddnr8334-sudo/inframon>

1. 이 프로그램이 하는 일

위성 레이더(Sentinel-1 InSAR)로 **교량의 미세 변위**를 재고, 그 관측을 물리 모델(PINN)에 넣어 **관측점이 없는 곳까지 변위장**을 채우고, 그것으로 **위험도(CRI)**를 내고, 전부를 **3D 디지털 트윈(IFC)** 위에 올립니다. 사용자가 하는 것은 **교량을 고르는 것**뿐입니다.

단계	무엇을	어디서
① 교량 선정	좌표 하나 → 연장·경간·폭·형식·형하고	파트너 실측 CSV → OSM → 표준데이터
② SLC 확보	Sentinel-1 SLC 검색·다운로드	ASF (Earthdata 토큰 필요)
③ InSAR	간섭도 → 위상 언래핑 → 변위 시계열	SNAP + snaphu
④ PS/DS	교량 위 산란체 선별, 쉬프트 보정	inframon
⑤ 잔차고도	점이 지면이 아니라 교면 위임 을 고도로 증명	inframon (수직기선 B _⊥)
⑥ IFC 트윈	실측 제원 → IFC4 → 점을 부재에 결합 → 3D	inframon
⑦ PINN-CRI	가상센싱(전체 변위장) → 위험도·경보	inframon
⑧ 브리프	건기연 형식 4 단 그림 (a)(b)(c)(d)	inframon
⑨ 감사	이 결과를 보고에 써도 되는가 — 파일이 스스로 판정	inframon
⑭ BMAP	파트너 플랫폼(Pontifex) 전송	HTTP API

2. 전체 흐름



각 단계는 **없는 것은 없다고 적고 넘어갑니다** — 멈추지 않습니다. 어느 교량을 보고에 쓸지는 감사 판정과 결과.md 를 보고 **유저가 정합니다**.

3. 다른 컴퓨터에서 처음부터 — PowerShell 단계별

아래 순서 그대로 하면 됩니다. 각 단계에 **확인** □ 이 있으니 그것이 나오면 다음으로, 안 나오면 6 장(막히면)으로. 처음 한 번은 10~20 분, 그다음부터는 대시보드만 띄우면 됩니다.

단계	하는 것	시간	확인
3.1	프로그램을 둘 폴더로 이동 → 한 줄 붙여넣기 (받기·파이썬 패키지·SNAP·snaphu 전부)	10~20 분	`결과: snap <input checked="" type="checkbox"/> , snaphu <input checked="" type="checkbox"/> , earthdata <input checked="" type="checkbox"/> , slc_dir <input "="" checked="" type="checkbox"/> Earthdata 토큰 확인(CMR 200)`
3.3	↳ 그 안에서 SLC 보관 폴더 고르기	10 초	` <input checked="" type="checkbox"/> SLC 보관 폴더 — E:\SLC`
3.4	이 PC 에 뭐가 있나 (진단)	10 초	[외부 도구·자격] 네 줄 전부 ☒
3.5	대시보드 띄우기 — 화면으로 보고 누르기	20 초	브라우저에 http://localhost:8501
3.6	먼저 되는 것으로 한 번 (정자교 시연)	44 초	`@ 결과 문서` · `rc=0` · 창 4 개
3.7	새 교량 — 계획만	1 분	`②④ SLC·트랙·프레임 ... N 장`
3.8	새 교량 — 끝까지 (SLC 다운로드 → InSAR → 트윈)	1~3 시간	`docs\bridges\<교량>\결과.md`
3.9	결과 보기	—	twin.viewer.html 더블클릭

3.0 미리 있어야 하는 것

	무엇	없으면
Python 3.11+	python.org — 설치 화면에서 'Add python.exe to PATH' 체크	3.1 의 한 줄이 winget 으로 갈아 줌
Git	git-scm.com	3.1 의 한 줄이 winget 으로 갈아 줌 (그것도 안 되면 zip 으로 받음)
Earthdata 계정	https://urs.earthdata.nasa.gov/users/new — 무료, 3 분, 메일 인증	사람이 직접 가입해야 함 — 프로그램이 대신 못 하는 유일한 것
디스크	SLC 1 장 4~8 GB × 교량당 30~50 장 → 200~400 GB	큰 드라이브를 3.3 에서 고른다

SNAP·snaphu·토큰 저장·SLC 폴더는 3.1 한 줄이 받아서 준비합니다. 트랙이 이미 있는 교량(정자교·청양교·내곡교 등)만 볼 거면 Python·Git 만 있으면 됩니다.

3.1 폴더로 이동 → 한 줄 (10~20 분)

PowerShell 을 엽니다(Win + X → 터미널). **먼저 프로그램을 둘 폴더로 이동합니다** — 그 아래에 `inframoon` 폴더가 생깁니다. 그다음 한 줄을 붙여넣고 Enter.

```
cd E:\
프로그램을 둘 곳 (원하는 드라이브·폴더)
```

①

```
irm https://raw.githubusercontent.com/tjddnr8334-sudo/inframon/main/install.ps1 | iex #
② 전부
```

이 한 줄이 순서대로:

- Python-Git 확인 — 없으면 winget 으로 설치
- GitHub 에서 받기 → `E:\inframon` (이미 있으면 git pull 로 갱신)
- 가상환경 `.venv` 만들고 파이썬 패키지 전부(torch-scipy-pyproj-rasterio-asf_search-streamlit 등) 설치 — 5~10 분
- **SNAP** 1.1 GB 내려받아 무인 설치 (이미 있으면 건너뛴) · **snaphu** WSL 안에 설치 (WSL 없으면 설치 걸고 재부팅 안내)
- **Earthdata** 토큰 — 브라우저를 열고 붙여넣기 기다림 → 3.2
- **SLC** 보관 폴더 — 드라이브를 묻는다 → 3.3
- 데모 → 진단 → 브라우저에 대시보드 <http://localhost:8501> (고려면 Ctrl + C)

손으로 하려면 (같은 폴더에서, 한 줄과 같은 일):

```
cd E:\
git clone https://github.com/tjddnr8334-sudo/inframon
cd inframon
python start.py --full --tools
```

확인 ☐ `결과: snap ☒, snaphu ☒, earthdata ☒, slc\_dir ☒` 와 그 아래 진단의 `판정: ☒ 코어 동작 가능`.

이후 명령은 `python` 이 아니라 `.venv\Scripts\python` 으로 부릅니다 — start.py 가 시스템 파이썬이 아니라 `.venv` 안에 설치하기 때문입니다. 그냥 `python -m inframon` 은 새 컴퓨터에서 'No module named inframon' 이 납니다. (`python start.py ...` 만 예외 — 이 파일은 표준 라이브러리만 씁니다.)

3.2 Earthdata 토큰 — 프롬프트가 뜨면 (1 분)

터미널에 `3) 여기 붙여넣고 Enter` 가 뜨고 브라우저에 Earthdata 페이지가 열립니다.

- 계정이 없으면 가입(무료): <https://urs.earthdata.nasa.gov/users/new> → 메일 인증 → 로그인
- 프로필 화면(이름·Username·Email 이 보임) 위쪽 작은 메뉴 **Generate Token** → 아래 **GENERATE TOKEN** 버튼
- 가려진 토큰 옆 **Show Token** 또는 복사 아이콘 → `eyJ0eXAiOi...` 로 시작하는 긴 문자열 **전체** 복사
- PowerShell 로 돌아와 붙여넣고 Enter — 프로그램이 NASA 서버(CMR)에 한 번 조회해 **유효할 때만** 저장

- 토큰은 비밀번호와 같습니다 — **터미널에만** 붙여넣고 채팅·메일·문서에는 넣지 않습니다.
노출됐으면 같은 페이지에서 새로 발급(최대 2 개)하고 옛것은 Revoke
- 60 일짜리 — 만료 7 일 전부터 프로그램이 자동 갱신하므로 다시 붙여넣을 일은 거의 없습니다

확인 ☒ Earthdata 토큰 확인(CMR 200) → C:\Users\<이름>\.inframondearthdata_token`.
그냥 Enter 로 건너뛰었으면 나중에 `python start.py --tools --no-demo`.

3.3 SLC 보관 폴더 — 물으면 (10 초)

위성 원본은 장당 4~8 GB, 교량 하나에 200~400 GB 까지 갑니다. C: 에 두면 곧 차니 **큰 드라이브**를 고릅니다. 드라이브별 여유 공간이 표시됩니다.

```
드라이브 여유 공간:
C:\  여유   120.3 GB / 476 GB
E:\  여유  1827.0 GB / 1863 GB
SLC 보관 폴더 (Enter = E:\SLC, 건너뛰려면 '-'): E:\SLC
```

원하는 경로를 치거나 Enter(여유가 가장 큰 드라이브의 `WSLC`). 이미 정해져 있으면 현재 위치를 보여 주고 **Enter = 유지, 새 경로 = 변경**. 이후 다운로드 는 `E:\WSLC\<궤도\_프레임>` 에 떨어지고, 같은 프레임을 쓰는 다음 교량은 다운로드를 건너뛴다. 나중에 바꾸려면 `.venv\Scripts\python -m inframon --slc-dir D:\WSLC` (없으면 만들).

확인 ☒ SLC 보관 폴더 — E:\WSLC (새로 만들) 또는 `유지: ...`.

3.4 이 PC 에 뭐가 있나 (10 초)

```
.venv\Scripts\python -m inframon --doctor

[외부 도구·자격 - `python start.py --tools` 가 준비하는 것]
☒ Earthdata 토큰   C:\Users\<이름>\.inframondearthdata_token (만료 D-59)
☒ SNAP gpt      C:\Program Files\esa-snap\bin\gpt.exe
☒ snaphu       wsl:/usr/bin/snaphu
☒ SLC 보관 폴더   E:\SLC (0 장)
[가능한 기능] ... ☒ slc_download ☒ insar_snap ☒ unwrap_snaphu ☒ full_pipeline
판정: ☒ 코어 동작 가능
```

확인 ☐ [외부 도구·자격] 네 줄 전부 ☒, [가능한 기능] 에 `full\_pipeline ☒`. **✗** 가 있으면 그 줄에 적힌 대로 하거나 `python start.py --tools --no-demo` (빠진 것만 다시).

3.5 대시보드 띄우기 (20 초)

명령 대신 화면으로 하려면 대시보드를 띄웁니다. 3.1 의 한 줄 설치기는 끝에 자동으로 띄우지만, `--tools` 나 `--full` 만 돌렸으면 뜨지 않으니 따로 띄웁니다:

```
python start.py --dashboard # 브라우저 자동 열림 (설치
상태도 같이 확인)
.venv\Scripts\streamlit run src\inframon\dashboard\app.py # 다음부터 대시보드만 바로
```

브라우저가 자동으로 열립니다. 안 열리면 주소창에 **http://localhost:8501** 을 직접 칩니다. 고려면 그 터미널에서 **Ctrl + C**.

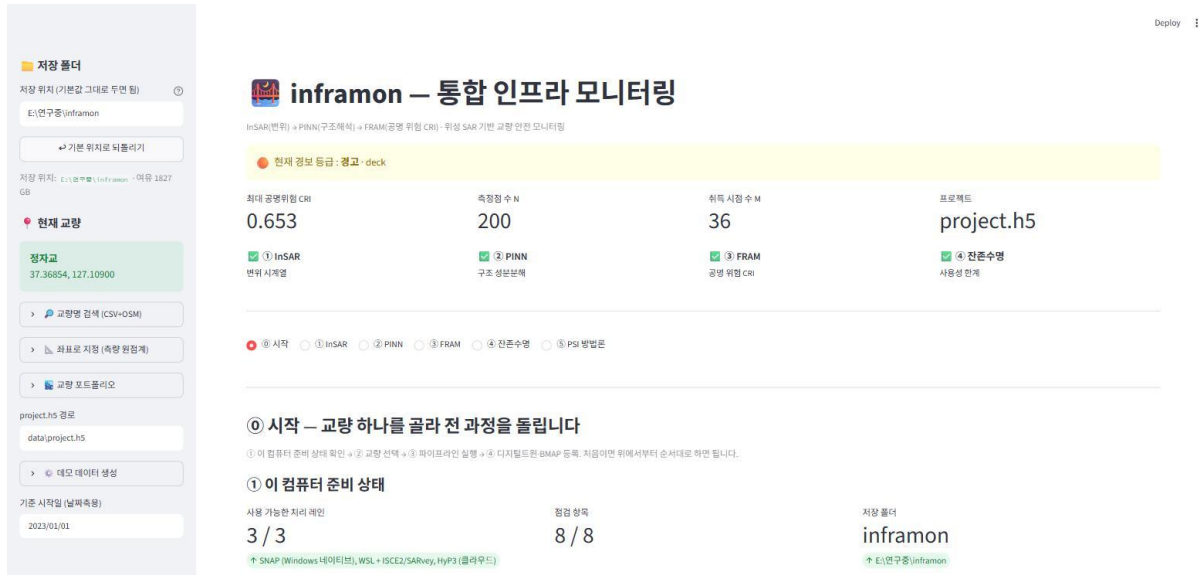


그림 1. 대시보드 — 왼쪽에서 교량을 고르고, 본문 ①→②→③ 순으로 누른다.

순서	화면	누를 것
①	이 컴퓨터 준비 상태	보기만. '모든 항목 준비 완료' 면 아래로. 막힌 항목이 있으면 🗑️ 외부 도구 준비 필침에서 버튼/붙여넣기
②	교량 선택	교량명 입력 → 🔍 찾기 (또는 위도·경도 칸에 직접 입력)
③	전 과정 실행	먼저 📅 계획 보기 (10~30 초, SLC 몇 장인지 확인) → 그다음 ▶ 전체 실행 (1~3 시간)
또는	탭 ① InSAR → ② PINN → ③ FRAM → ④ 잔존수명	한 단계씩 보며 가려면 각 탭 맨 위 ▶ 이 단계 실행. 앞 단계 결과가 없으면 놀리지 않고 무엇을 먼저 할지 알려 줍니다. 탭 줄 아래 진행 띠에 단계별 ☑/☐

3.6~3.8 의 명령줄과 같은 일을 이 세 단계가 합니다. 진행 상황에 `②④ SLC·트랙·프레임 — ASC path127 · 41 장` 처럼 ☑ 가 찍히면 그 교량은 돌릴 수 있는 것입니다. 왼쪽 사이드바 🔍 교량명 검색 → 지도 마커 클릭 → 📅 타깃 저장 → 🗑️ 끝까지 돌리기 로 해도 같습니다.

탭의 뜻: ① 시작 은 준비 확인·교량 선택·실행. ① InSAR · ② PINN · ③ FRAM · ④ 잔존수명 은 그 순서로 만들어지는 결과를 보는 곳이고, 각 탭 맨 위 ▶ 로 그 단계만 (다시) 돌릴 수도 있습니다 — ①

의 ▶ 전체 실행과 같은 코드, 같은 결과 파일(<저장 폴더>WpipelineWproject.h5). ⑤ **PSI 방법론** 은 PS·SBAS·QPS 세 처리법 비교(참고).

확인 □ 브라우저에 그림 1 화면. ① **이 컴퓨터 준비 상태** 가 '모든 항목 준비 완료'.

3.6 먼저 되는 것으로 한 번 — 정자교 시연 (44 초)

```
.venv\Scripts\python scripts\demo_4pm.py
```

이미 처리된 트랙으로 정자교를 끝까지 돌리고 결과 창 4 개(3D 속도 · 3D 위험도 · 브리프 그림 · 결과.md)를 엽니다. 이것이 되면 ⑥~⑩(트윈·PINN·브리프·감사·결과)이 이 PC 에서 도는 것입니다.

확인 □ 터미널에 ⑩ **결과 문서** 까지 찍히고 `rc=0`. 3D 창에서 드래그 회전, 점 클릭 = 값·부재.

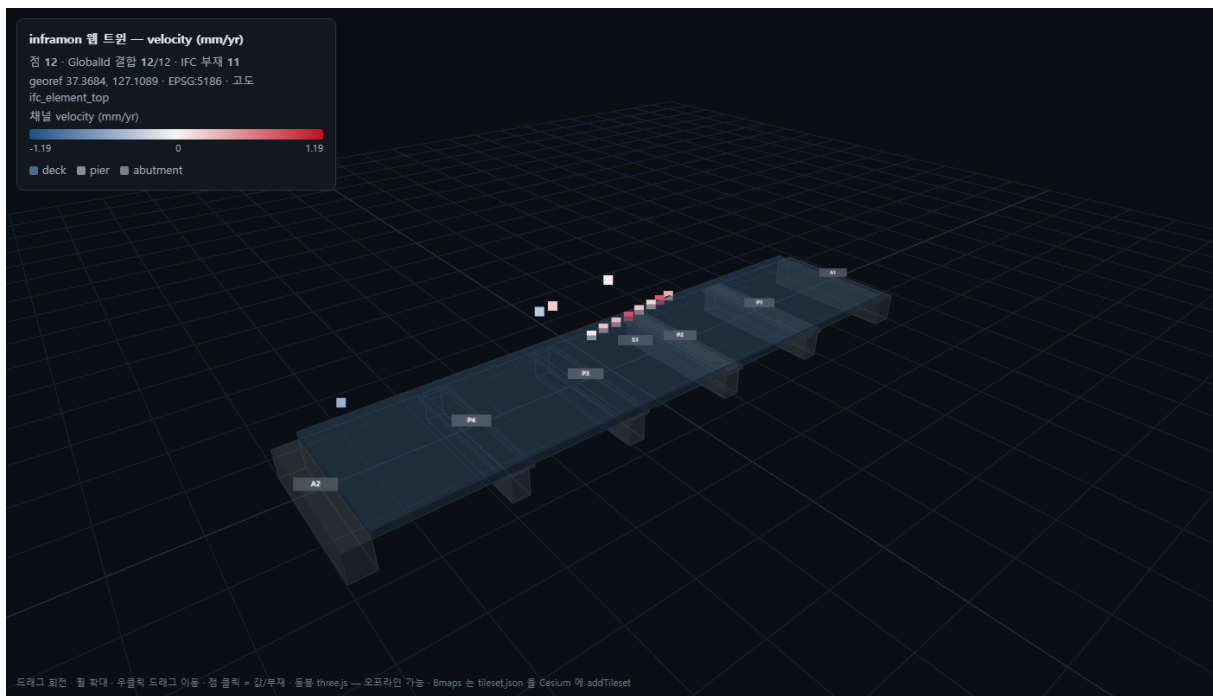


그림 2. 3D 디지털 트윈 — IFC 부재(A1·P1~P4·S1) 위에 InSAR 점. 점을 클릭하면 값·부재·GlobalId.

3.7 새 교량 — 계획만 먼저 (1 분)

```
.venv\Scripts\python -m inframon --pipeline 37.5337,126.9366 --pipeline-mode plan --out docs\bridges\마포대교\plan
```

좌표만 주면 제원(파트너 CSV → OSM → 표준데이터)과, 어느 궤도에 SLC 가 몇 장 있는지를 검색합니다. 다운로드 는 하지 않습니다. 장면 수가 10 장 미만이면 그 교량은 이 궤도로 어렵습니다 — 다른 교량이나 궤도를 봅니다.

확인 □ ②④ **SLC·트랙·프레임 ASC path127 frame120 · 41 장** 처럼 궤도·프레임·장면 수. ①의 ✕ 는 OSM 서버 일시 오류(504) — 다시 돌리면 됩니다.

3.8 새 교량 — 끝까지 (1~3 시간)

```
.venv\Scripts\python scripts\bridge_run.py --name 마포대교 --lat 37.5337 --lon 126.9366
```

트랙이 없으므로 ⑩ SLC 다운로드(3.3 폴더로) → SNAP → 언래핑부터 갑니다. 단계마다 찍합니다:

⑩ SLC → InSAR ← 대부분의 시간 (다운로드 GB 단위 · SNAP 수십 분)
① 제원 ② 테크선 ③ 지면 ④ 점 선택 ⑤ 잔차고도
⑥ IFC 트윈 ⑦ PINN·CRI ⑧ 브리프 ⑨ 감사 ⑩ 결과 문서

중간에 죽으면 같은 명령을 다시 — 받아 둔 SLC 는 재사용합니다. 여러 교량은 `--batch docs\bridges\batch.json`.

확인 □ `docs\bridges\마포대교\결과.md` — 각 단계가 무엇을 어디서 가져왔는지, 못 한 것은 왜인지. 4 장에서 읽는 법.

3.9 결과 보기

파일 (docs\bridges<교량>W)	무엇	여는 법
twin.viewer.html	3D 트윈(변위 속도 채널)	더블클릭 — 인터넷 불필요
twin_cri.viewer.html	3D 트윈(CRI 위험도 채널)	더블클릭
brief.png	건기연 형식 4 단 그림 (a)(b)(c)(d)	더블클릭
결과.md	수치 표 · 출처 · 감사 판정 · 적어 둘 것 · 원리상 못 하는 것	메모장 · VS Code
<교량>_proxy.ifc	IFC4 프록시 교량	Revit · BlenderBIM

화면으로 보려면 3.5 의 대시보드 — 왼쪽 포트폴리오에서 교량을 고르면 위 탭 ① InSAR · ② PINN · ③ FRAM 에 같은 결과가 나옵니다.

4. 결과를 어떻게 읽나

4.1 브리프 그림 (a)(b)(c)(d)

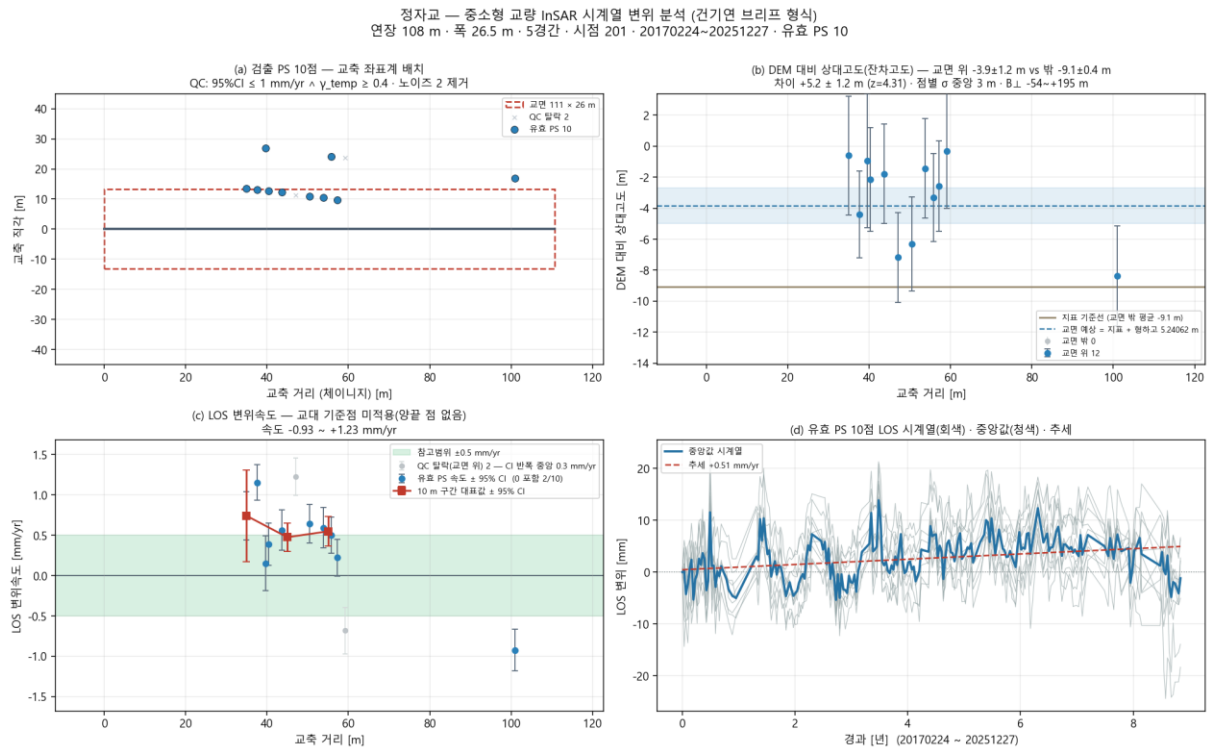


그림 3. 정자교 — (a) PS 교축 배치 (b) 잔차고도: 교면 위 점이 지면보다 $+5.2 \pm 1.2$ m (c) 속도 $\pm 95\%$ CI (d) 시계열·추세

패널	보는 것	판단
(a)	PS 가 교면 안에 몇 개, 어디에	교면 밖 점은 지반 — 교량 값에 섞이면 안 됨
(b)	DEM 대비 상대고도	교면 위 점이 지면보다 형하고만큼 높으면 교면 위 산란체 확인
(c)	속도 $\pm 95\%$ CI, ± 0.5 mm/yr 참고범위	오차막대가 0 을 포함 → 유의한 변형 없음
(d)	시계열 중앙값 · 추세	계절 주기 정상 · 추세 mm/yr

4.2 감사 판정

판정	뜻	예
☒ 보고 가능	연래핑·교량 포함·경간·PINN 물리값 전부 통과	청양교 · 내곡교
☐ 조건부	쓸 수는 있으나 사유가 있음 — 사유를 함께 적을 것	정자교(붕괴 교량, 관측으로 못 봄)
☒ 보고 불가	래핑 위상 · 교량 위 점 0 · 비물리 값	동수원(OSM 연장 불일치)

① 표기는 감점이 아니라 반드시 알아야 할 사실입니다 — 예: '타·고유진동수는 관측값이 아니라 설계 제원 기반'.

4.3 PINN 가상센싱

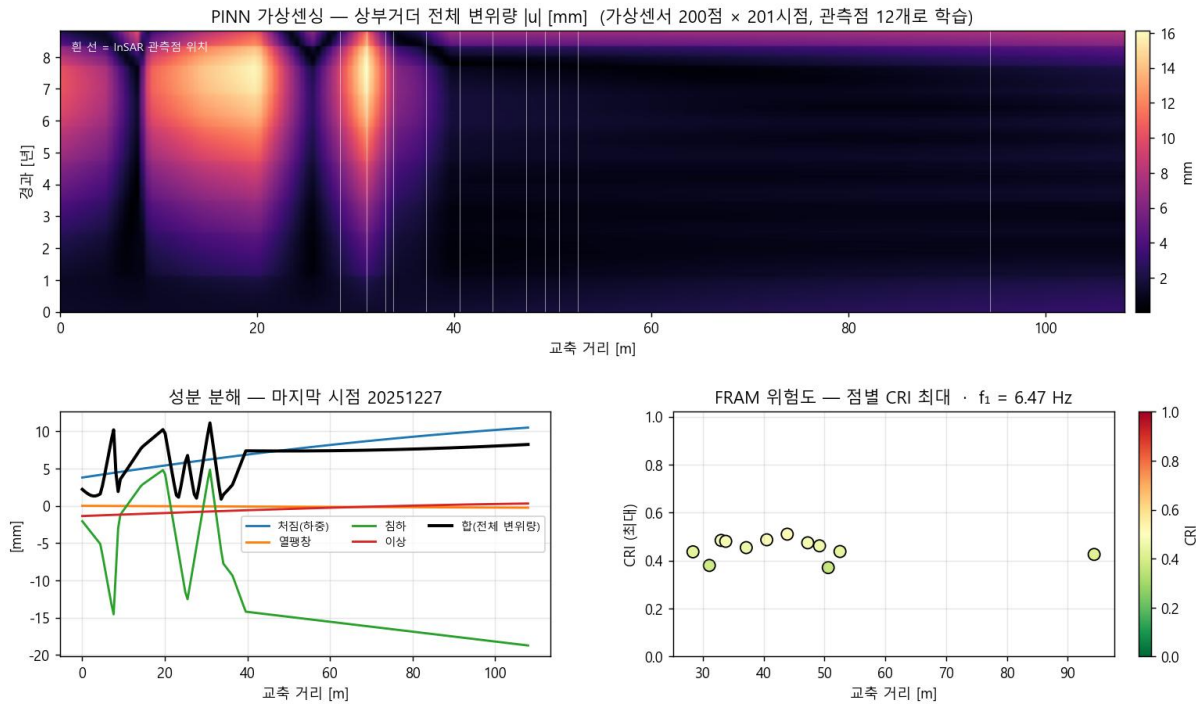


그림 4. 관측점 12 개로 학습한 PINN 이 교축 200 점 × 201 시점의 전체 변위장을 채운다. 흰 선 = 관측점 위치. 관측점 밖은 외삽.

5. 이 프로그램이 원리상 못 하는 것 — 결과를 읽기 전에

항목	왜	그러면
EI(강성) 관측 식별	InSAR 는 상대 변위 — 자중 처짐(청양교 이론 345 mm)은 위성이 보기 전에 이미 들어가 있음	타·고유진동수는 설계 제원 기반 (① 표기). 절대 강성은 레벨링·GNSS 필요
데크 위 PS 밀도	Sentinel-1 화소 ~11 m. 81 m 교량은 교축 구간당 화소 1.3 개	고해상도 SAR·코너리플렉터. 긴 교량(마포대교 1,390 m)은 유리
쉬프트 방향	궤도 heading 이 정함 — 조정 대상 아님	크기 δh 는 잔차고도로 관측값 대체
국부 붕괴 탐지	정자교 2023-04-05 보도부 붕괴(수 m)는 화소보다 작음. 열·추세 제거 후 계단 $z = -0.6$	알려진 사고 교량은 자동 표기하고 '정상'을 보고 근거로 쓰지 않음
시점 수	속도 95% CI 는 장 수가 정함. 25 장 → 2.5 mm/yr, 201 장 → 0.2 mm/yr	브리프 기준 ≥ 100 장. 부족하면 판정 보류

6. 막히면

- ① '토큰 없음' → 3.2 로. 발급은 `urs.earthdata.nasa.gov` (1 분).
- 'SNAP gpt 없음' → SNAP 설치 후 PATH 또는 환경변수 SNAP_HOME.
- 'snaphu 없음' → WSL 에 snaphu. 언래핑이 실패하면 파라미터 사다리 4 단계가 자동으로 돌고, 그래도 안 되면 `unwrap_retry.json` 에 사유.
- OSM 504 → 자동 재시도 3 회. 캐시(`osm_roads_500m.json`)가 있으면 캐시.
- 중간에 죽음 → 같은 명령 다시. 받은 SLC 는 재사용. 결과.md 에 '예외:' 줄이 남음.
- '교면 위 점 0' → 교량이 작거나 궤도 방향이 불리함. 계획(3.7)에서 장면 수·궤도를 먼저 볼 것.

7. 지금까지 돌린 교량 (docs/bridges/)

교량	점	CRI	잔차고도(교면-지면)	감사	비고
청양교	44	0.784	+12.5 ± 6.4 m	보고 가능	25 시점 — 속도 CI 넓음
정자교	12	0.569	+5.2 ± 1.2 m (z 4.3)	조건부	2023-04-05 붕괴 교량 — 관측으로 못 봄
내곡교	34	0.821 경고	—	보고 가능	건기연 브리프 같은 교량 (PS 35 vs 32)
칠백로	22	0.808 경고	—	조건부	
상규	3	0.879 위험	—	보고 가능	점 3 개 — 판정 신뢰도 낮음
동수원	77	0.729	—	보고 불가	CSV 890 m vs OSM 1227 m

여러 교량을 한 번에: ``.\venv\Scripts\python scripts\bridge_run.py --batch docs\bridges\batch.json``